

2023 데이터 크리에이터 캠프 DATA CREATOR CAMP

대학부 네트로닉스

팀장 이민성

박민선

석대근

김완수



목차

MISSION1

- 문제 및 데이터 이해
- ResNet 18 모델
- 수행 과정 및 결과
- 고찰

MISSION2

- 문제 이해
- 수행 과정 및 결과 (정성적, 정량적 분석)
- 고찰

MISSION3

- 문제 이해
- 수행 과정 및 결과
- 고찰

1. MISSION 1 – 문제 및 데이터 이해

문제의 이해

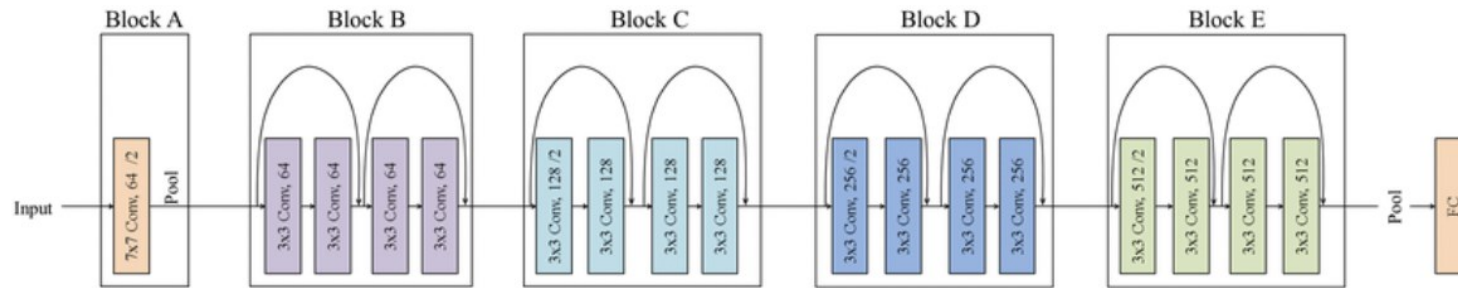
- MISSION 1의 1-1은 각 다른 해상도의 컬러를 가지는 한국 이미지 데이터셋을 불러와 클래스별로 하나씩 총 42장을 한번에 plot하여 확인하는 것이다. 따라서 파일처리와 이미지 데이터에 대한 전처리가 요구된다.
- MISSION 1의 1-2는 ResNet18모델을 활용하여 42종의 클래스 분류를 수행하고 Validation data에 대해 정확도(Accuracy)를 제시하는 것이다. 기본적으로 ResNet18의 모델과 딥러닝 학습의 전반적인 이해가 필요하다.

1. MISSION 1 – 문제 및 데이터 이해

데이터의 이해

- MISSION 1,2 에 사용되는 데이터 셋은 42개의 클래스를 가지는 한국 음식 이미지 데이터이며 학습용 이미지(train)와 검증용 이미지(val)가 있다.
- Train data set을 통하여 ResNet18 모델을 학습시키고 validation data set 으로 학습한 모델을 검증하여 validation 데이터에 대한 정확도를 제시할 수 있다.

1. MISSION 1 – ResNet18 모델

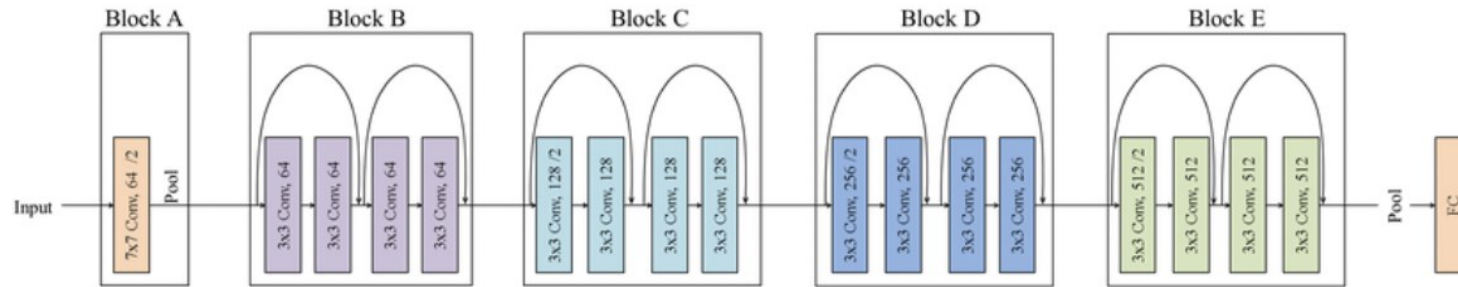


<ResNet-18의 구조>

ResNet 18 model의 Input은 224x224x3 Image로 들어간다. 따라서 Image를 Resize, Normalize 등을 적용시켜 차원을 맞춘다.

4개의 Convolution block, FC(Fully Connected) layer로 구성되어 있는데 42개의 class가 출력 되어야 하므로 FC layer의 output features를 수정한다.

1. MISSION 1 – ResNet18 모델



<ResNet-18의 구조>

위의 ResNet을 학습시켜야 하므로 parameters를 초기화 시키고 직접 학습을 통해 가중치를 업데이트한다.

일정 epoch만큼 학습시킨 ResNet18을 이용하여 Validation data에 대한 정확도 검증을 통해 모델의 성능을 확인한다.

1. MISSION 1-1. 수행 과정 및 결과

1-1-1. DATA LOADER

```
path = 'C:/Users/kfood_train/train'
path_val = 'C:/Users/kfood_val/val'

label_list = os.listdir(path)
label_val_list = os.listdir(path_val)
label_only_list = label_list
label_list = sorted(label_list)
```

```
label_dict = {
    label_list : i for i, label_list in enumerate(label_list)
}
label_val_dict = {
    label_val_list : i for i, label_val_list in enumerate(label_val_list)
}
```

데이터 셋이 저장된 폴더 경로를 지정하고 각 클래스의 이름을 변수에 저장한다.

변수에 클래스 이름이 잘 저장되었는지 알아보기 위해 dictionary 형태의 변수를 만들고 클래스 이름을 순서대로 print 하여 확인한다.

1. MISSION 1-1. 수행 과정 및 결과

1-1-1. DATA LOADER

```
print(len(label_list))  
print(label_dict)
```

```
42  
{'갈비구이': 0, '갈치구이': 1, '감자전': 2, '경단': 3, '계란국': 4, '계란말이': 5, '계란후라이': 6,
```

Print 결과, label_list에 저장된 클래스 이름이 42개 있는 것과
순서대로 클래스 이름이 정확히 저장된 것을 확인할 수 있었다.

1. MISSION 1-1. 수행 과정 및 결과

1-1-1. DATA LOADER

```
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
```

이미지 데이터를 불러오기 전 model input에 맞는 차원 데이터가 들어가야 한다.

Resize, ToTensor, Normalize를 사용하여 전처리를 적용시킬 수 있는 변수를 생성한다.

1. MISSION 1-1. 수행 과정 및 결과

1-1-1. DATA LOADER

```
dataset = datasets.ImageFolder(root = path, transform = transform)
dataset_val = datasets.ImageFolder(root = path_val, transform = transform)

trainloader = DataLoader(dataset, batch_size = batch_size, shuffle=True, num_workers=2)
trainloader_val = DataLoader(dataset_val, batch_size = batch_size, shuffle=False, num_workers=2)
```

ImageFolder 함수를 사용하여 경로에 있는 이미지 데이터 셋을 전처리하여 불러온다.

그 후 DataLoader 함수를 통해 이미지 데이터를 load할 수 있는 변수를 생성한다. 학습용 데이터 셋의 경우 무작위로 학습시키기 위해 데이터를 shuffle하여 load 하도록 한다.

1. MISSION 1-1. 수행 과정 및 결과

1-1-2. PLOT

```
fig, ax = plt.subplots(6, 7, figsize=(10, 8))
```

```
for i in label_only_list:  
    img_path = path + "/" + i + "/"  
    label_list_2 = os.listdir(img_path)  
    random_plot = random.choice(label_list_2)
```

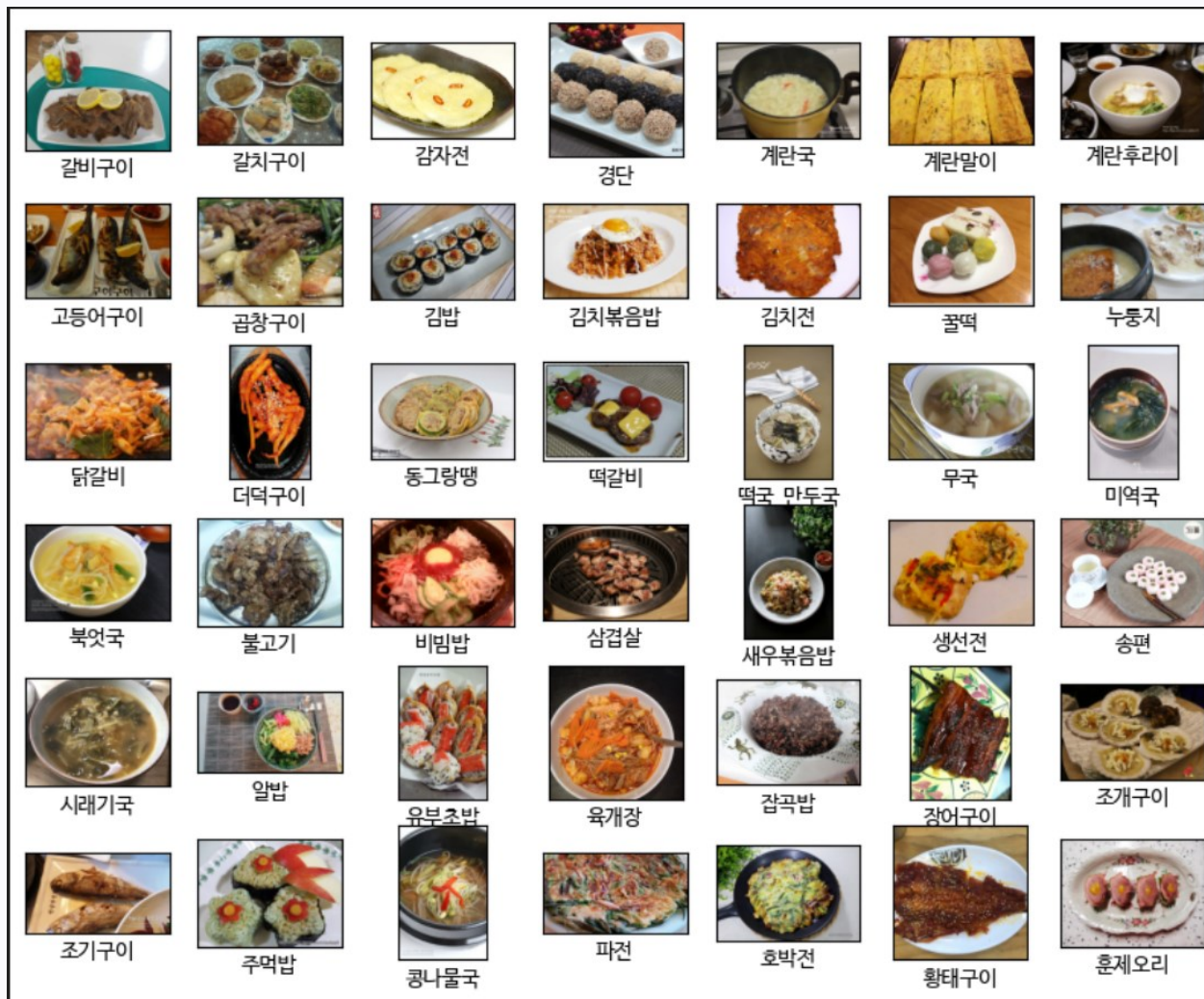
```
random_img_path = img_path + random_plot  
print(random_img_path)  
random_img = mpimg.imread(random_img_path)  
ax[row, column].imshow(random_img)
```

path에 저장된 이미지를 랜덤하게 42개의 클래스 마다 각 한 개씩 출력하도록 하였고
Matplotlib 라이브러리를 import하여 이미지를 시각화 하였다.

1. MISSION 1-1. 수행 과정 및 결과

MISSION 1-1 결과

6x7 size로 각 42개의 클래스마다
random하게 image를 plot한 결과이다.
42개의 클래스 모두 이미지가 하나씩 잘
plot 된 것을 확인할 수 있다.



1. MISSION 1-2. 수행 과정 및 결과

1-2-1. ResNet18 setting

```
model_1 = models.resnet18(pretrained = False)
fc_in_features = model_1.fc.in_features
model_1.fc = nn.Linear(fc_in_features, len(label_list))
model_1 = model_1.to(device)
```

Pretrained 된 ResNet18 모델을 사용하지 않기 위해 pretrained = False 옵션을 사용하였다. 또한 input features 는 ResNet18의 default 값을 사용, output이 42이므로 output features를 42로 설정하여 학습이 가능한 모델로 setting한다.

1. MISSION 1-2. 수행 과정 및 결과

1-2-1. ResNet18 setting 확인

```
...  
    )  
    )  
    (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))  
    (fc): Linear(in_features=512, out_features=42, bias=True)  
    )
```

수정한 ResNet18 model을 확인한 결과,

마지막 fc layer의 output features가 default 값인 1000에서 42로 바뀐 것을 볼 수 있었다.

1. MISSION 1-2. 수행 과정 및 결과

1-2-2. Training

```
epoch = 50
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(resnet_pt.parameters(), lr=0.001, momentum=0.9)

if j % 20 == 19:    # print every 20 mini-batches
    print(f'[{i + 1}, {j + 1:5d}] example_loss: {running_loss_20 / 20 :.3f}')
    running_loss_20 = 0.0
```

Epoch=50으로 지정하고 손실함수는 CrossEntropyLoss, optimizer는 빠르게 학습시키기 위해 SGD(Stochastic Gradient Descent)를 사용하였다.

또한 1epoch내 20 mini-batch 마다 loss를 출력하여 경향성을 직관적으로 파악하기 쉽게 설정하였다.

1. MISSION 1-2. 수행 과정 및 결과

1-2-2. Training 결과1

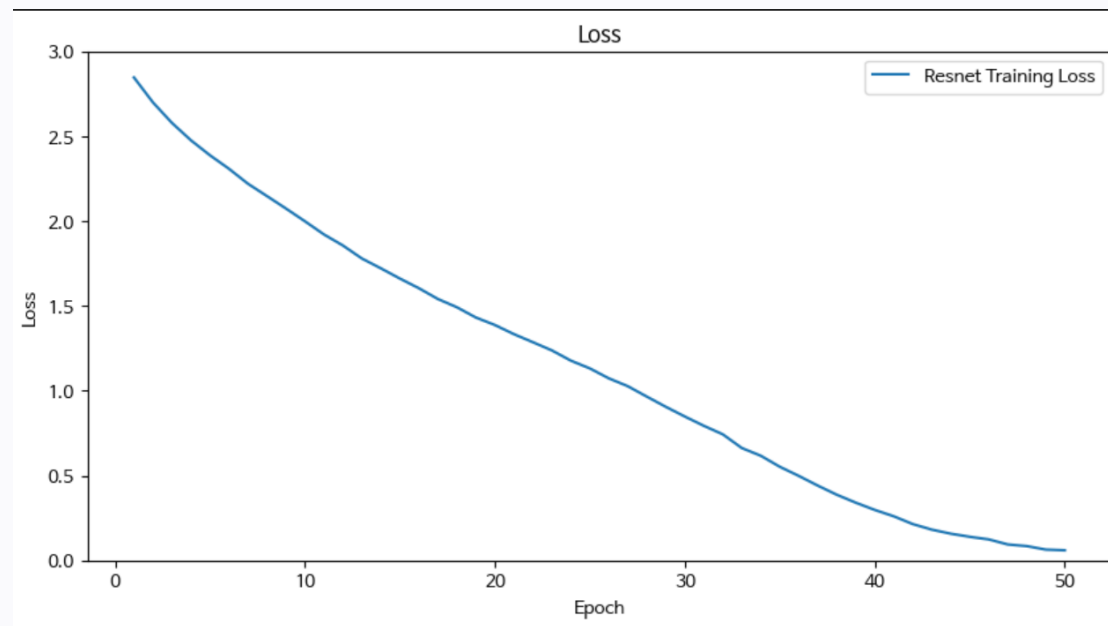
Training 결과, 일반적으로 epoch
가 증가할 때마다 loss가 감소하
는 경향을 보였다.

Epoch가 후반으로 갈 수록 일정
값에 수렴하는 것을 볼 수 있었다.

```
... [1, 20] example_loss: 3.028
[1, 40] example_loss: 2.938
[1, 60] example_loss: 2.946
[1, 80] example_loss: 2.928
[1, 100] example_loss: 2.923
[1, 120] example_loss: 2.890
[1, 140] example_loss: 2.833
[1, 160] example_loss: 2.905
[1, 180] example_loss: 2.842
[1, 200] example_loss: 2.822
[1, 220] example_loss: 2.848
[1, 240] example_loss: 2.752
[1, 260] example_loss: 2.865
[1, 280] example_loss: 2.819
[1, 300] example_loss: 2.794
[1, 320] example_loss: 2.869
[1, 340] example_loss: 2.815
[1, 360] example_loss: 2.803
[1, 380] example_loss: 2.796
[1, 400] example_loss: 2.856
[1, 420] example_loss: 2.740
[1, 440] example_loss: 2.866
[1, 460] example_loss: 2.839
[1, 480] example_loss: 2.807
[1, 500] example_loss: 2.788
...
[50, 520] example_loss: 0.064
Epoch [50/50], Loss: 0.0600
checkpoint save
Finished Training
```

1. MISSION 1-2. 수행 과정 및 결과

1-2-2. Training 결과2



이를 시각화 하여 그래프로 출력하였고 어떠한 tuning도 하지 않고 한번의 학습을 거친 ResNet18의 Loss 경향을 쉽게 확인할 수 있다.

1. MISSION 1-2. 수행 과정 및 결과

1-2-3. Save model

```
PATH = 'mission1.pt'  
torch.save(model_1.state_dict(), PATH)  
print('Final_Save Model Success')
```

학습을 통해 가중치가 업데이트 된 ResNet18 모델을 'mission1.pt'라는 이름으로 저장한다. 'state_dict()'메서드를 사용하여 가중치 및 bias 를 저장하고 나중에 불러올 수 있다.

1. MISSION 1-2. 수행 과정 및 결과

1-2-4. Validation

```
@torch.no_grad()
def validate_epoch(model: nn.Module, data_loader: DataLoader, device: torch.device):
    ''' data_loader provides inputs and GTs.
        model receives input images from data_loader and produces logits.
        returns the accuracy of model in percent.
    '''
    model.eval()
```

학습 시킨 ResNet model을 검증 시 위와 같이 주어진 Validation 코드를 사용하였다. Model.eval() 함수를 사용하여 검증 시 가중치가 업데이트 되지 않도록 한다. 위 validate_epoch 사용자 정의 함수를 사용시 return 값으로 accuracy를 받는다.

1. MISSION 1-2. 수행 과정 및 결과

MISSION 1-2 결과

```
validation_accuracy=validate_epoch(model_1, trainloader_val, device)
print(validation_accuracy) # Validation accuracy print
```

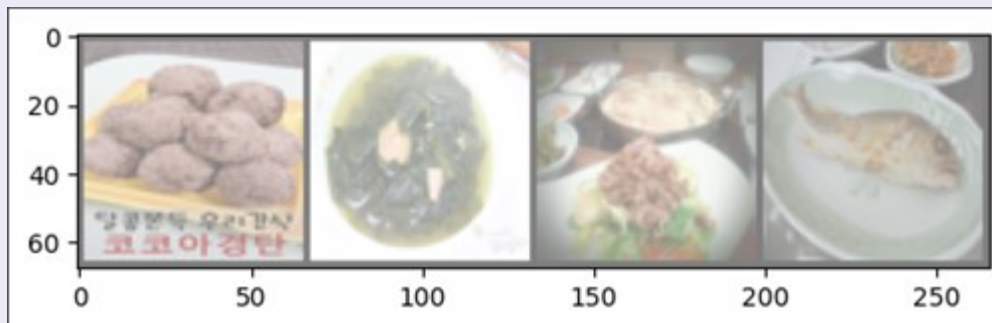
```
69.6998519897461
```

검증 이미지 데이터를 사용하여 50Epoch 만큼 한 번 학습시킨 ResNet18 모델의 정확도를 validation 한 결과 약 69.7 %가 나왔다.

1. MISSION 1 고찰

정규화(Normalization)

1. 정규화(Normalization) 문제 : 초기에 data를 load시 pytorch의 내장함수를 사용하지 않고 data를 load하는 class를 따로 만들어서 데이터를 불러오고 정규화를 하였다. 그러나 정규화가 제대로 적용되지 않는 어려움을 겪었다. 그 결과 정규화가 제대로 되지 않는데 정규화를 해제하고 이미지를 plot하여 다음과 같이 컬러 이미지가 뚜렷하게 나오지 않는 경우가 발생하였다.

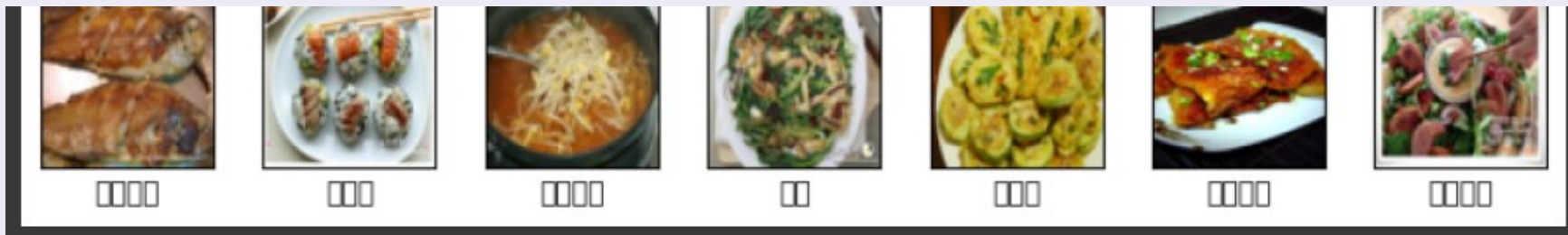


Pytorch의 내장함수인 ImageFolder 함수와 DataLoader 를 사용하여 위의 문제를 해결할 수 있었다.

1. MISSION 1 고찰

폰트

2. 폰트 문제 : 42개의 image를 plot 결과 다음과 같이 한글이 제대로 출력되지 않는 문제가 발생하였다.



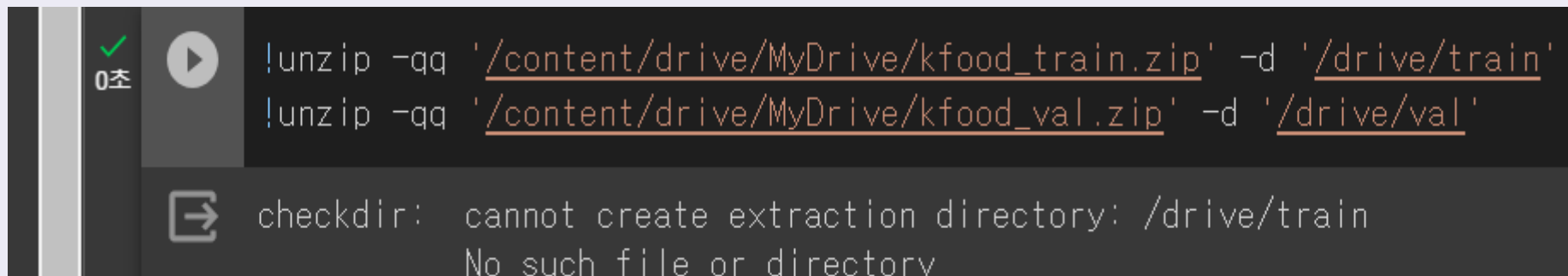
한글 폰트가 없다는 것을 인지하고 폰트를 다운하는 코드를 작성하여 위 문제를 해결하였다. 이를 통해 파이썬에서 image plot시 한글이 기본으로 지원하지 않는다는 사실을 알 수 있었다.

```
!sudo apt-get install -y fonts-nanum  
!sudo fc-cache -fv  
!rm ~/.cache/matplotlib -rf
```

1. MISSION 1 고찰

파일, 경로

3. 파일, 경로 문제 : colab에서 cell에서 오류가 나거나 최초로 실행 시 첫번째 cell부터 실행시켰는데 데이터 셋을 zip형식으로 경로에 두었기 때문에 매번 unzip을 하여 zip을 해제한 파일을 만들어야 했다. Unzip cell을 실행할 때마다 새로운 파일이 계속 생성이 되고 이로 인해 파일 경로가 꼬였다.



```
✓ 0초 ▶ !unzip -qq '/content/drive/MyDrive/kfood_train.zip' -d '/drive/train'
!unzip -qq '/content/drive/MyDrive/kfood_val.zip' -d '/drive/val'
⏏ checkdir: cannot create extraction directory: /drive/train
No such file or directory
```

Zip 형식이 아닌 폴더 형식의 파일을 경로에 두었고 unzip의 과정 없이 쉽게 이미지 데이터나 label을 load할 수 있었다.

1. MISSION 1 고찰

GPU

4. GPU 문제 : Colab Pro를 사용하여 5epoch 실행 시 runtime이 최소 2시간 이상 걸리고 colab pro 자체에서 일정시간 이상 사용 시 제한을 거는 문제가 발생하였다. 따라서 노트북 local 환경으로 바꾸어 실행하였으나 GPU 관련 문제인 CUDA error가 발생하였다. 이 error로 인해 학습을 시키는 과정이 매우 오래 걸리는 어려움을 겪었다.

```
RuntimeError: CUDA error: unknown error
CUDA kernel errors might be asynchronously reported at some other API call, so the stacktrace below
For debugging consider passing CUDA_LAUNCH_BLOCKING=1.
Compile with `TORCH_USE_CUDA_DSA` to enable device-side assertions.
```

성능이 우수한 하드웨어의 local에서 실행하여 위 문제를 해결하였다.

2. MISSION 2 – 문제 이해

문제의 이해

- MISSION 2의 2-1은 Mission 1의 결과를 분석하여 맞춘 샘플과 틀린 샘플의 차이점과 각 공통점을 분석하여 모델이 어떤 특징 예측에 강점이고 약점인지 파악한다.
- MISSION 2의 2-2는 위의 Mission에서의 분석을 기반으로 모델의 성능을 높이기 위해 ResNet 아키텍처의 종류를 변경하거나 신경망의 하이퍼 파라미터를 조절한다.
- MISSION 2의 2-3은 가장 높은 정확도의 모델 setting을 찾기 위하여 수행한 과정과 결과를 논리적으로 따지고 최종 결과를 도출해낸다.

2. MISSION 2-1. 수행 과정 및 결과 – 정성적 분석

2-1-1. Confusion matrix

```
for images, labels in tqdm(data_loader, total=len(data_loader), mininterval=1, desc='measuring accuracy'):
    images = images.to(device)
    labels = labels.to(device)

    print(labels[0])

    predictionary['target_batch'].append(labels.tolist())
    predictionary['target'] = np.concatenate(predictionary['target_batch']).tolist()

    logits = model(images)
    pred = torch.argmax(logits, dim=1)

    predictionary['predicted_batch'].append(pred.tolist())
    predictionary['predicted'] = np.concatenate(predictionary['predicted_batch']).tolist()

    accuracies.append(pred == labels)
```

confusion matrix를 구현하기 위하여, validation dataset으로 accuracy를 측정할 때, 각 epoch마다 target(정답)과 predicted(예측)의 list를 만든다. 이 때 각각의 값은 배치사이즈(64)만큼의 numpy 배열이므로, 리스트로 변환 후 1차원 배열로 바꾸어 list에 추가하도록 한다.

2. MISSION 2-1. 수행 과정 및 결과 – 정성적 분석

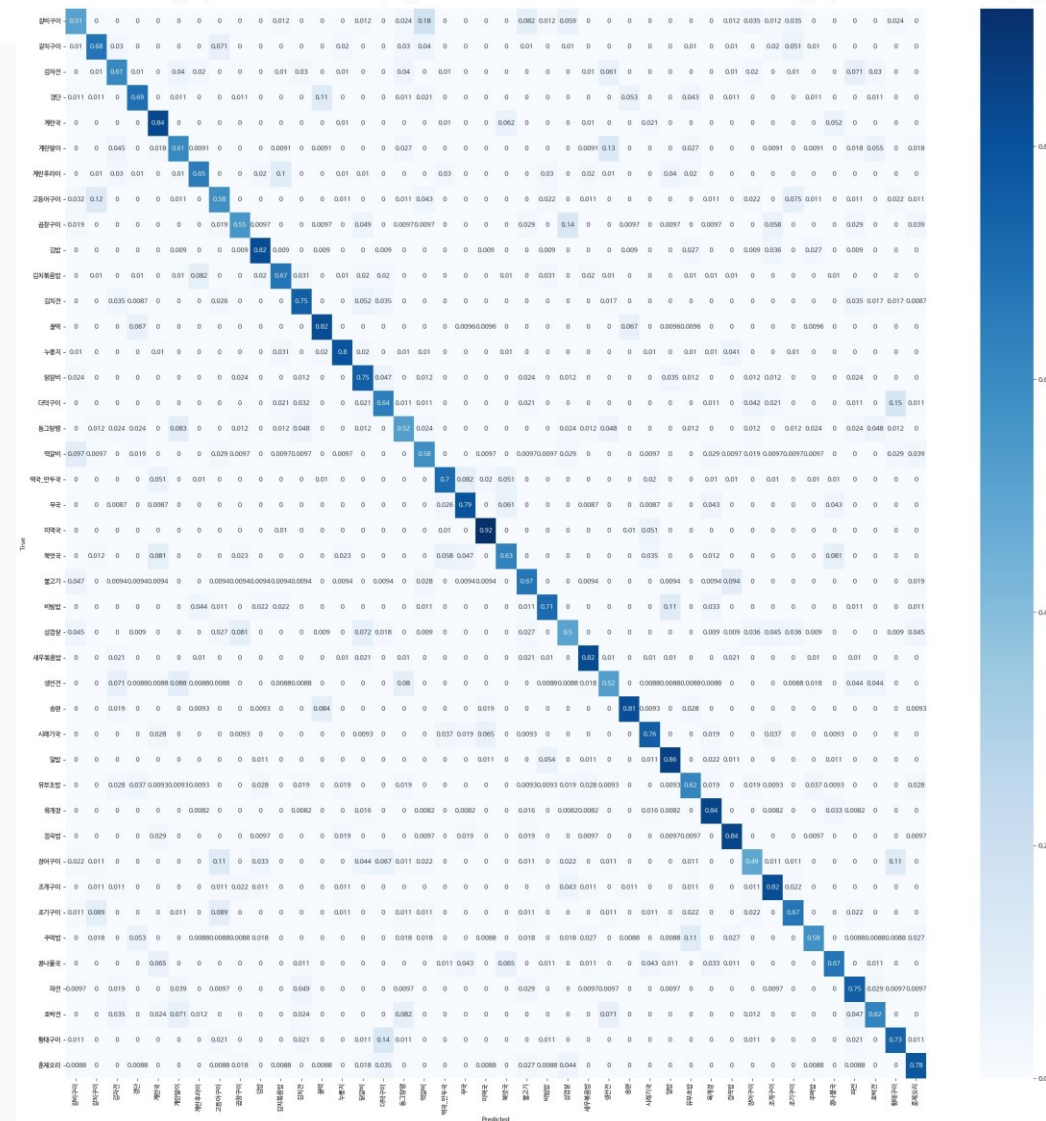
2-1-1. Confusion matrix

```
label_list = os.listdir(path)
label_list_cm = sorted(label_list)

y_target= []
y_predicted = []
y_target = predictionary['target']
y_predicted = predictionary['predicted']

cm = confusion_matrix(y_target, y_predicted, normalize = 'true')
cm_col_row = pd.DataFrame(cm / np.sum(cm, axis=1)[: , None],
index = [i for i in label_list_cm], columns = [i for i in label_list_cm])
plt.figure(figsize = (30,30))
sns.heatmap(cm_col_row, annot = True, cmap = 'Blues')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()
```

Pytorch에서 제공하는 confusion_matrix 함수를 이용하여, 실제 정답인 원소로 이뤄진 y_target과 예측한 원소로 이루어진 y_predicted를 넘겨주면 confusion matrix를 plot할 수 있다.



2. MISSION 2-1. 수행 과정 및 결과 – 정성적 분석

2-1-1. Confusion matrix

갈비구이	0.51	0	0	0	0	0	0	0	0	0	0.012	0	0	0	0.012	0	0.024	0.18
갈치구이	0.01	0.68	0.03	0	0	0	0	0.071	0	0	0	0	0	0.02	0	0	0.03	0.04
감자전	0	0.01	0.61	0.01	0	0.04	0.02	0	0	0	0.01	0.03	0	0.01	0	0	0.04	0
경단	0.011	0.011	0	0.69	0	0.011	0	0	0.011	0	0	0	0.11	0	0	0	0.011	0.021
계란국	0	0	0	0	0.84	0	0	0	0	0	0	0	0	0.01	0	0	0	0
계란말이	0	0	0.045	0	0.018	0.61	0.0091	0	0	0	0.0091	0	0.0091	0	0	0	0.027	0
계란후라이	0	0.01	0.03	0.01	0	0.01	0.65	0	0	0.02	0.1	0	0	0.01	0.01	0	0	0
고등어구이	0.032	0.12	0	0	0	0.011	0	0.58	0	0	0	0	0	0.011	0	0	0.011	0.043
곰창구이	0.019	0	0	0	0	0	0	0.019	0.55	0.0097	0	0	0.0097	0	0.049	0	0.0097	0.0097
김밥	0	0	0	0	0	0.009	0	0	0.009	0.82	0.009	0	0.009	0	0	0.009	0	0

위 그림은 confusion matrix의 일부를 가져온 것이다. 결과를 분석해보면 김밥과 계란국 class를 비교적 잘 맞춘다는 것을 알 수 있고 갈비구이 class는 0.5근처의 value를 가지며 다른 클래스로 예측할 확률이 많이 분포한다는 것을 볼 수 있다. 전체 결과를 보았을 때 미역국 class를 맞출 확률이 0.92로 제일 높았고 장어구이 class를 맞출 확률이 0.49로 제일 낮았다.

2. MISSION 2-1. 수행 과정 및 결과 – 정성적 분석

2-1-1. Confusion matrix

갈비구이	0.51	0	0	0	0	0	0	0	0	0	0.012	0	0	0	0.012	0	0.024	0.18
갈치구이	0.01	0.68	0.03	0	0	0	0	0.071	0	0	0	0	0	0.02	0	0	0.03	0.04
감자전	0	0.01	0.61	0.01	0	0.04	0.02	0	0	0	0.01	0.03	0	0.01	0	0	0.04	0
경단	0.011	0.011	0	0.69	0	0.011	0	0	0.011	0	0	0	0.11	0	0	0	0.011	0.021
계란국	0	0	0	0	0.84	0	0	0	0	0	0	0	0	0.01	0	0	0	0
계란말이	0	0	0.045	0	0.018	0.61	0.0091	0	0	0	0.0091	0	0.0091	0	0	0	0.027	0
계란후라이	0	0.01	0.03	0.01	0	0.01	0.65	0	0	0.02	0.1	0	0	0.01	0.01	0	0	0
고등어구이	0.032	0.12	0	0	0	0.011	0	0.58	0	0	0	0	0	0.011	0	0	0.011	0.043
곰창구이	0.019	0	0	0	0	0	0	0.019	0.55	0.0097	0	0	0.0097	0	0.049	0	0.0097	0.0097
김밥	0	0	0	0	0	0.009	0	0	0.009	0.82	0.009	0	0.009	0	0	0.009	0	0

Value가 0.8이상인 class의 특징들을 분석한 결과 음식의 특징 중에 진한 검은색 계열의 색이 포함되어있거나 김밥이나 떡, 미역국 의 [떡(원) 밥 알갱이(타원), 미역(사각형)] 처럼 비슷한 도형이 연속적으로 배열되어 있다고 판단하였다. 반면 value가 0.5근처의 값을 가지는 class는 색 조합이 다양하고 연하다는 특징을 가지고 있다고 판단하였다.

2. MISSION 2-1. 수행 과정 및 결과 – 정성적 분석

2-1-2. 틀린 샘플 및 맞은 샘플 plot

틀린 샘플과 맞은 샘플들을 15개씩 plot하기 위해서 2-1-1. confusion matrix에서 이용했던 predicted와 target을 numpy로 바꾸고 np.where를 이용하여 y_predicted_np와 y_target_np의 값이 틀리면, 인덱스를 wrong 리스트에 저장하고 맞으면 correct 리스트에 저장한다.

```
for i in label_list:
    img_path = path_val + '/' + i + '/'
    img_list_perClass = os.listdir(img_path)

    for j in range(len(img_list_perClass)):
        img_path_total = img_path + img_list_perClass[j]
        img_list.append(img_path_total)

y_predicted_np = np.array(predictionary['predicted'])
y_target_np = np.array(predictionary['target'])

wrong = np.where(y_predicted_np != y_target_np)
wrong_list = list(wrong[0])

correct = np.where(y_predicted_np == y_target_np)
correct_list = list(correct[0])

random_wrong_list = random.sample(wrong_list, 15)
random_correct_list = random.sample(correct_list, 15)
```

2. MISSION 2-1. 수행 과정 및 결과 – 정성적 분석

예측과 정답이 맞았을 때

2-1-3. 예측 성공한 sample 특징

Mission1에서 학습시킨 모델로 validation 이미지를 test해보았을 때 정답을 맞춘 샘플들의 공통점은 명확히 일반화 할 수는 없지만 색이 진하고 단조롭다는 특징이 있다고 판단하였다.



실제 정답: 떡갈비
예측한 값: 떡갈비



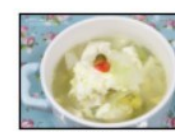
실제 정답: 유부초밥
예측한 값: 유부초밥



실제 정답: 닭갈비
예측한 값: 닭갈비



실제 정답: 계란말이
예측한 값: 계란말이



실제 정답: 계란국
예측한 값: 계란국



실제 정답: 장어구이
예측한 값: 장어구이



실제 정답: 유부초밥
예측한 값: 유부초밥



실제 정답: 꿀떡
예측한 값: 꿀떡



실제 정답: 조기구이
예측한 값: 조기구이



실제 정답: 육개장
예측한 값: 육개장



실제 정답: 감자전
예측한 값: 감자전



실제 정답: 장어구이
예측한 값: 장어구이



실제 정답: 곰탕구이
예측한 값: 곰탕구이



실제 정답: 무국
예측한 값: 무국



실제 정답: 송편
예측한 값: 송편

2. MISSION 2-1. 수행 과정 및 결과 – 정성적 분석

예측과 정답이 틀렸을 때

2-1-3. 예측 실패한 sample 특징

예측에 실패한 샘플들은 색이 연하고 비교적 다채롭다는 특징이 있다고 판단하였다. 또한 특정 클래스의 음식을 구분하지 못하는 것이 아닌, 같은 클래스라도 색이 연하고 다채롭다면 예측에 실패할 확률이 높다는 것을 확인하였다.



실제 정답: 복엇국
예측한 값: 시래가국



실제 정답: 김치볶음밥
예측한 값: 김밥



실제 정답: 계란국
예측한 값: 콩나물국



실제 정답: 파전
예측한 값: 호박전



실제 정답: 곰탕구이
예측한 값: 불고기



실제 정답: 유부초밥
예측한 값: 주먹밥



실제 정답: 생선전
예측한 값: 동그랑땡



실제 정답: 계란후라이
예측한 값: 떡국, 만두국



실제 정답: 유부초밥
예측한 값: 주먹밥



실제 정답: 복엇국
예측한 값: 계란국



실제 정답: 장어구이
예측한 값: 김밥



실제 정답: 김치전
예측한 값: 떡볶이



실제 정답: 조기구이
예측한 값: 갈치구이



실제 정답: 갈비구이
예측한 값: 떡갈비



실제 정답: 감자전
예측한 값: 김치전

2. MISSION 2-1 – 수행 과정 및 결과 – 정량적 분석

사용한 코드

2-1-4. MISSION 1의 결과 분석 – 컨볼루션 필터 분석

```
model_2.to(device)
```

- MISSION 1에서 resnet 18 model을 기반으로 만든 모델에서의 컨볼루션 필터 크기를 확인했다.

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
  (layer1): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
    (1): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
)
```

```
(layer2): Sequential(
  (0): BasicBlock(
    (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
    (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
    (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (downsample): Sequential(
      (0): Conv2d(64, 128, kernel_size=(1, 1), stride=(2, 2), bias=False)
      (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (1): BasicBlock(
    (conv1): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
    (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  )
)
```

2. MISSION 2-1 – 수행 과정 및 결과 – 정량적 분석

2-1-4. MISSION 1의 결과 분석 – 컨볼루션 필터 분석

- 컨볼루션 필터 크기를 확인한 후 각 컨볼루션 필터의 가중치를 시각화했다.

Layers	컨볼루션 필터 크기
1-layers	64개
2-layers	64개
3-layers	128개
4-layers	256개
5-layers	512개

2. MISSION 2-1 – 수행 과정 및 결과 – 정량적 분석

2-1-4. MISSION 1의 결과 분석 – 컨볼루션 필터 분석

- 각 layer의 컨볼루션 필터의 가중치를 확인하는 코드

```
# 두 번째 레이어의 첫 번째 컨볼루션 레이어의 가중치 추출
second_conv_layer_weights = model_2.layer1[0].conv1.weight.data

# 이미지를 [0, 1] 범위로 정규화하는 변환 적용
normalize = transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])

# 시각화할 필터의 수 결정 (최대 64개)
num_filters_to_visualize = min(second_conv_layer_weights.shape[0], 64)
```

2. MISSION 2-1 – 수행 과정 및 결과 – 정량적 분석

2-1-5. MISSION 1의 결과 분석 – 컨볼루션 필터 가중치 시각화

- 각 layer의 컨볼루션 필터의 가중치를 시각화를 진행했고 가중치가 어떻게 분포되는지 분석했다.

```
# 첫 번째 컨볼루션 레이어의 가중치 추출
first_conv_layer_weights = model_2.conv1.weight.data

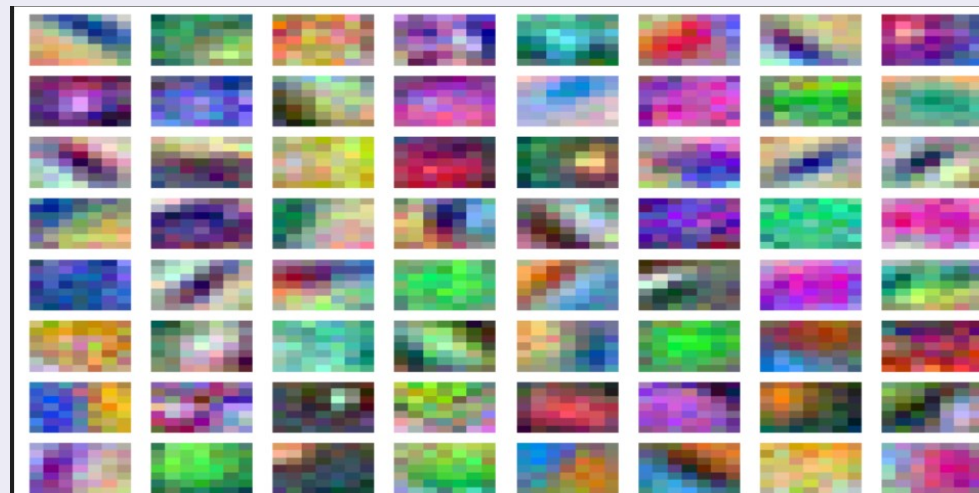
# 이미지를 [0, 1] 범위로 정규화하는 변환 적용
normalize = transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
preprocess = transforms.Compose([transforms.ToPILImage(), transforms.Resize((224, 224)), transforms.ToTensor(), normalize])

# 시각화할 필터의 수 결정
num_filters_to_visualize = first_conv_layer_weights.shape[0]
```

2. MISSION 2-1 – 수행 과정 및 결과 – 정량적 분석

2-1-6. MISSION 1의 결과 분석 – 1-layer 가중치 시각화 결과

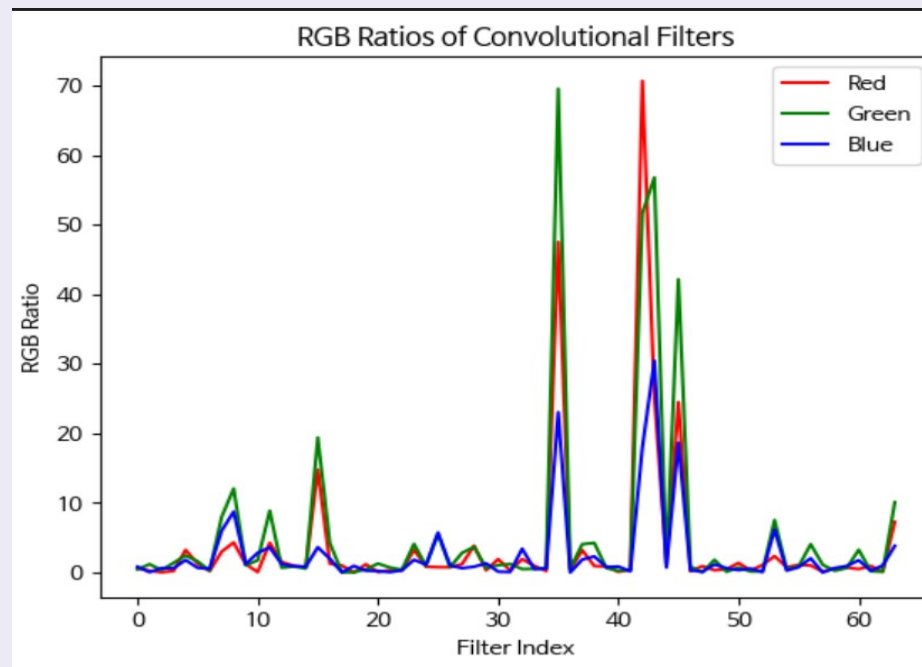
- 1-layer 가중치 시각화를 통해 컨볼루션 필터가 어느 위치에 가중치를 많이 두는지, 어느 색에 가중치를 많이 두는지 확인했다.



2. MISSION 2-1 – 수행 과정 및 결과 – 정량적 분석

2-1-7. MISSION 1의 결과 분석 – 1-layer 컨볼루션 필터의 RGB 분포

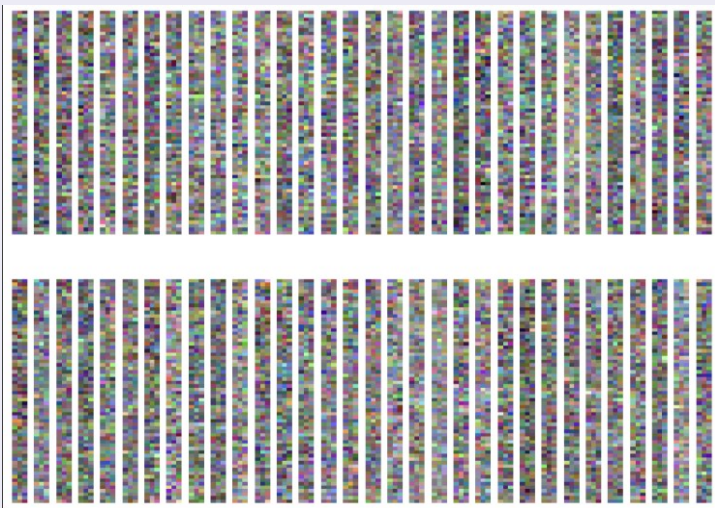
- 1-layer 컨볼루션 필터가 어느 색에 중점적으로 가중치를 부여하는지 확인하기 위해 각 컨볼루션 필터의 RGB 분포를 plot한 후 분석을 진행했다.



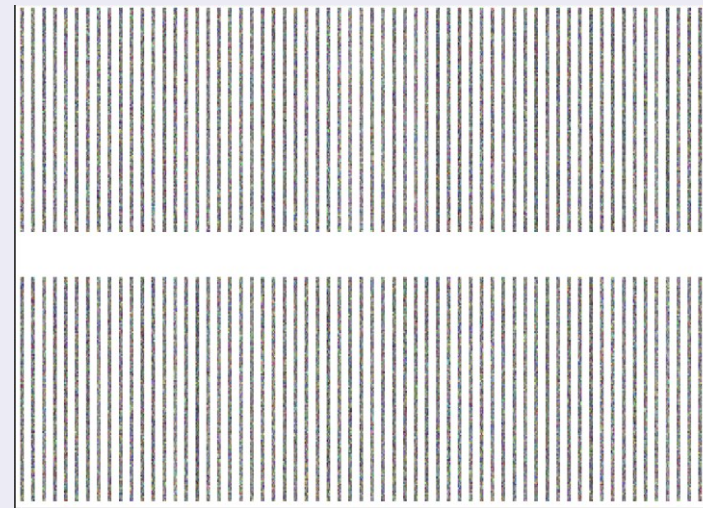
2. MISSION 2-1 – 수행 과정 및 결과 – 정량적 분석

2-1-8. MISSION 1의 결과 분석 – 1-layer 이외의 다른 layer에서의 가중치 시각화 결과

- 1-layer 이외의 다른 layer 컨볼루션 필터의 가중치를 시각화한 후 가중치를 분석하려고 했지만 필터의 크기가 1-layer에 비해 크므로 분석을 진행하지 않았다.



2-layer 컨볼루션 필터 가중치 시각화



3-layer 컨볼루션 필터 가중치 시각화

2. MISSION 2-2. 수행 과정 및 결과

2-2-1. Augmentation

```
train_augmentation = augmentations.Compose([
    augmentations.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1, p=1),
    augmentations.Rotate(p=1),
    augmentations.HorizontalFlip(p=1)
    # , augmentations.pytorch.transforms.ToTensorV2()
], p=1)
```



<https://gaussian37.github.io/dl-pytorch-augmentation/>

정확도를 높이기 위하여 과적합을 막는 augmentation을 진행했다. 그 중에서도 이미지를 다양하게 변형할 수 있는 augmentation을 사용했고, 2-1에서 색깔의 차이를 두어야 한다는 것을 알았기 때문에 ColorJitter함수를 사용했다. 또한, Resnet은 locality of pixel dependencies가 존재하기 때문에 이미지를 뒤집거나 회전시키도록 하였다.

2. MISSION 2-2. 수행 과정 및 결과

2-2-1. Augmentation

```
transformed = train_albumentation(image = image)
transformed_image = transformed["image"]
```

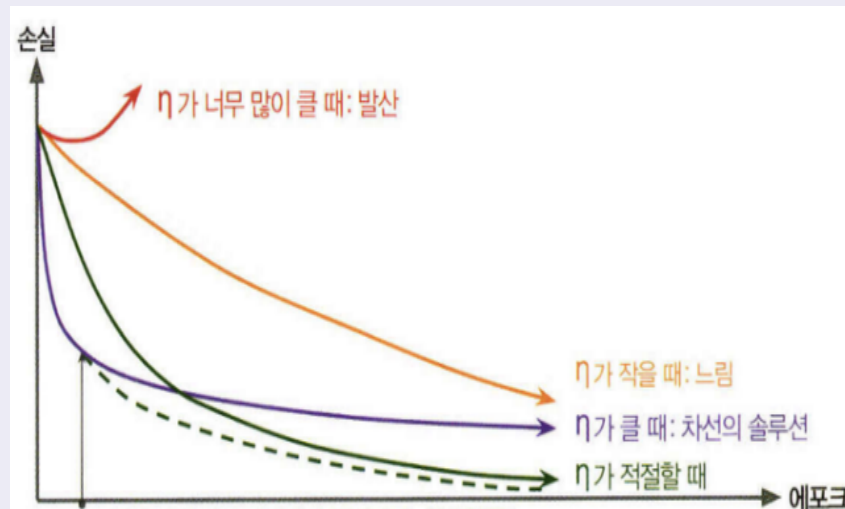
```
aug_name = aug_path + '/' + str(cnt) + '_' + str(j) + '.jpg'
print(aug_name)
cv2.imwrite(aug_name, transformed_image)
```

이미지를 불러와서 Augmentation을 진행하고 나서, 따로 aug_folder를 제작하여 augmentation한 데이터를 저장한다. 학습시킬 때는 aug_folder를 imagefolder 함수를 이용하여 부른 데이터와 기존의 dataset과 aug_folder를 concat하여 학습시킨다.

2. MISSION 2-2. – 성능 향상을 위한 작업

2-2-2. LR scheduling

- MISSION 1에서 추출한 모델보다 성능이 우수한 모델을 만들기 위해 LR scheduling을 활용했다.
- 학습률을 조절함으로써 최적의 가중치를 구하는 과정을 가졌다.



2. MISSION 2-2. – 성능 향상을 위한 작업

2-2-2. LR scheduling – 파라미터 조정

실험적으로 모델 학습을 진행한 후 accuracy와 loss plot을 통해 최적의 하이퍼파라미터 값을 찾는 과정을 가졌다.

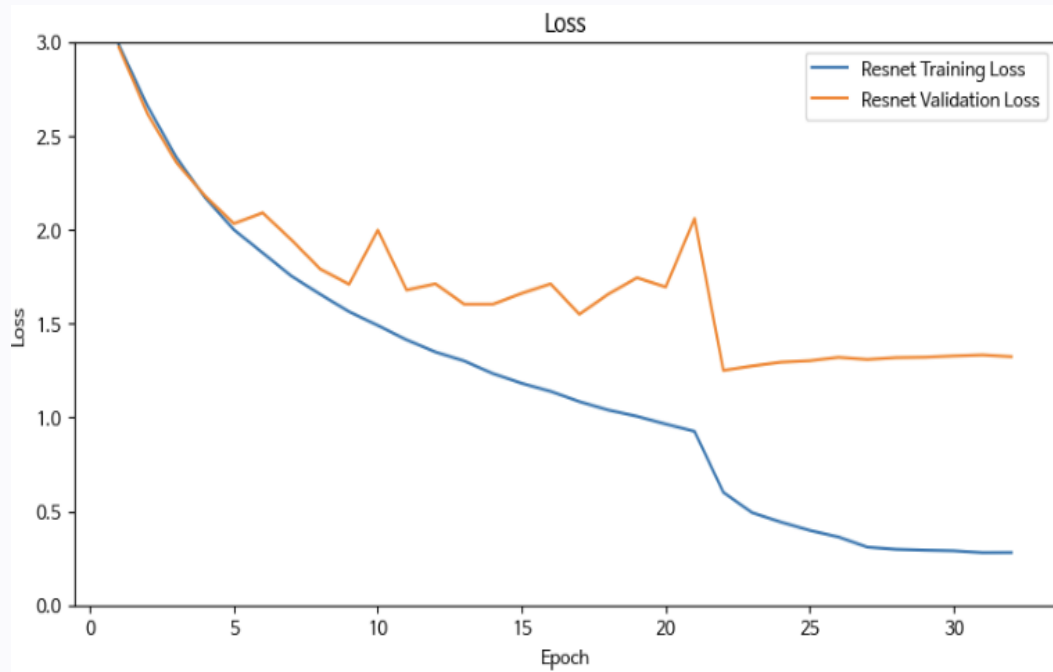
파라미터명	의미
mode	검증 손실값이 감소할 때 성능이 개선되었다고 판단
factor	Learning rate를 감소시킬 비율
Patience	N번의 연속적인 epoch 동안 검증 손실값이 개선되지 않으면 LR 감소
verbose	True 설정하면 감소된 LR에 대한 정보 출력

```
optimizer = optim.Adam(model_2.parameters(), lr=0.001, weight_decay=0.001)
scheduler = ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=3, verbose=True)
scheduler_train = ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=3, verbose=True)
```

2. MISSION 2-2. – 성능 향상을 위한 작업

2-2-2. LR scheduling – 하이퍼파라미터 찾는 과정

- Optimizer 함수와 scheduler의 파라미터를 조절하며 최적의 파라미터를 찾는 과정을 진행했다.



```
In [34]: @torch.no_grad()
def validate_epoch(model: nn.Module, data_loader: DataLoader, device: torch.device):
    """ data loader provides inputs and GTs.
        model receives input images from data_loader and produces logits.
        returns the accuracy of model in percent.
    """
    model.eval()
    accuracies = []
    for images, labels in tqdm(data_loader, total=len(data_loader), mininterval=1, desc='measuring accuracy'):
        images = images.to(device)
        labels = labels.to(device)

        logits = model(images)
        pred = torch.argmax(logits, dim=1)

        accuracies.append(pred == labels)

    accuracy = torch.concat(accuracies).float().mean() * 100
    return accuracy.item()
```

```
In [35]: validation_accuracy=validate_epoch(model_2, trainloader_val, device)
print(validation_accuracy) # Validation accuracy print
```

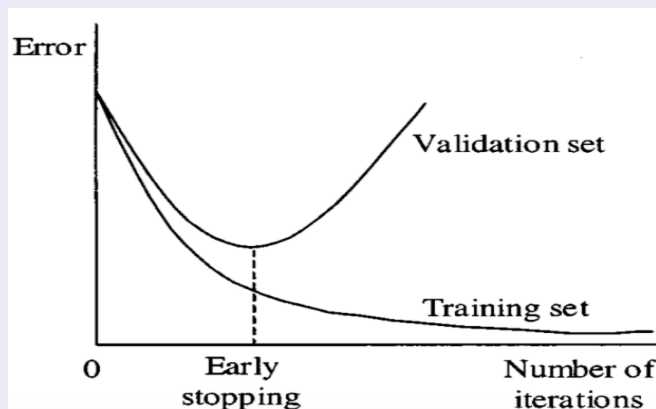
measuring accuracy: 100% 66/66 [00:14<00:00, 4.66it/s]

57.47975158691406

2. MISSION 2-2. – 성능 향상을 위한 작업

2-2-3. Early stopping

- LR 스케줄링을 진행해도 더 이상 loss가 감소하지 않는 경우가 발생하거나 loss가 오히려 증가하는 overfitting이 되는 경우가 발생한다. Overfitting이 발생하면 더 이상 학습을 진행해도 유의미한 결과를 얻지 못하므로 early stopping을 구현하여 loss의 변화를 통해 학습을 종료할지 더 진행할지 판단하였다.

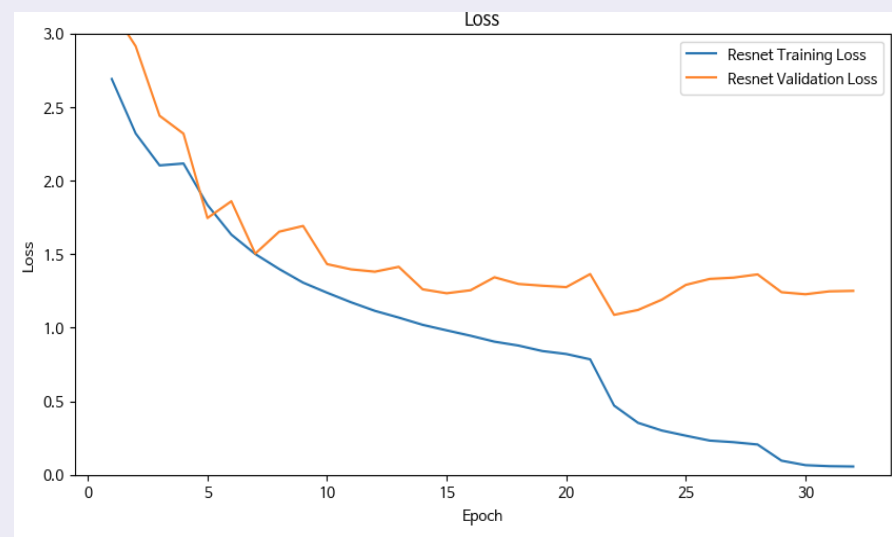


2. MISSION 2-2. – 성능 향상을 위한 작업

2-2-3. Early stopping – 파라미터 조정

- Early stopping patience를 10으로 설정한 결과 화면이다.
- 10번의 epoch 동안 검증 손실이 증가하는 경향을 보여 의미있는 학습 과정이 아니라고 판단하여 early stopping이 된 결과 이다.

```
Epoch [27/50], Training Loss: 0.0652  
Epoch [27/50], Validation Loss: 1.2270  
Current Counter: 8  
  
/home/pc-15/anaconda3/envs/pc15/lib/py  
ytes should be converted to RGBA image  
warnings.warn(  
/home/pc-15/anaconda3/envs/pc15/lib/py  
ytes should be converted to RGBA image  
warnings.warn(  
  
Epoch [28/50], Training Loss: 0.0582  
Epoch [28/50], Validation Loss: 1.2468  
Current Counter: 9  
  
/home/pc-15/anaconda3/envs/pc15/lib/py  
ytes should be converted to RGBA image  
warnings.warn(  
/home/pc-15/anaconda3/envs/pc15/lib/py  
ytes should be converted to RGBA image  
warnings.warn(  
  
Epoch [29/50], Training Loss: 0.0561  
Epoch [29/50], Validation Loss: 1.2501  
Current Counter: 10  
Early stopping at epoch 29
```



2. MISSION 2-2. – 성능 향상을 위한 작업

2-2-4. Regularization – weight decay

- 정확도를 증가시키는 방법 중 하나로 regularization이 있다. Overfitting을 방지하기 위해 weight decay를 진행했다. Weight decay는 모델의 weight의 제곱합을 패널티 term으로 주어 손실을 최소화했다. 즉, L2 regularization을 진행했다. 아래의 코드를 통해 가중치를 0.001씩 감소된다는 것을 의미한다.

```
optimizer = optim.Adam(model_2.parameters(), lr=0.001, weight_decay=0.001)
```

2. MISSION 2-3. 수행 과정 및 결과

2-3-1. 초기 하이퍼 파라미터 세팅 1

```
## L2 Regularization도 포함
optimizer = optim.SGD(model_2.parameters(), lr=0.01, weight_decay=0.001, momentum=0.9) #
scheduler = ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=5, verbose=True)
```

```
In [35]: validation_accuracy=validate_epoch(model_2, trainloader_val, device)
print(validation_accuracy) # Validation accuracy print
```

measuring accuracy: 100%  66/66 [00:14<00:00, 4.66it/s]

57.47975158691406

Name	Type	Description
brightness	float or tuple of float (min, max)	How much to jitter brightness. brightness_factor is chosen uniformly from [max(0, 1 - brightness), 1 + brightness] or the given [min, max]. Should be non negative numbers.
contrast	float or tuple of float (min, max)	How much to jitter contrast. contrast_factor is chosen uniformly from [max(0, 1 - contrast), 1 + contrast] or the given [min, max]. Should be non negative numbers.
saturation	float or tuple of float (min, max)	How much to jitter saturation. saturation_factor is chosen uniformly from [max(0, 1 - saturation), 1 + saturation] or the given [min, max]. Should be non negative numbers.
hue	float or tuple of float (min, max)	How much to jitter hue. hue_factor is chosen uniformly from [-hue, hue] or the given [min, max]. Should have 0 <= hue <= 0.5 or -0.5 <= min <= max <= 0.5.

초기에는 lr을 0.01, scheduler factor = 0.1, albumentation의 hue = 0.1로 설정하였다. 하지만 정확도가 낮아 발산하는 것을 막기 위해 lr를 0.01로 설정하고, albumentation을 찾아본 결과 hue의 범위가 -0.5 ~ 0.5였기 때문에 hue를 0.05로 바꾸기로 하였다.

2. MISSION 2-3. 수행 과정 및 결과

2-3-1. 초기 하이퍼 파라미터 세팅 1

```
## L2 Regularization도 포함
optimizer = optim.SGD(model_2.parameters(), lr=0.1, weight_decay=0.001, momentum=0.9) # 옵티마이저
scheduler = ReduceLROnPlateau(optimizer, mode='min', factor=0.4, patience=5, verbose=True)
scheduler_train = ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=5, verbose=True)

validation_accuracy=validate_epoch(model_2, trainloader_val, device)
print(validation_accuracy) # Validation accuracy print

measuring accuracy: 100%|██████████| 66/66 [00:14<00:00, 4.49it/s]
59.74273681640625
```

기존의 코드에서 Lr을 0.1, hue = 0.05로 바꿨을 때이다. 기존보다는 정확도가 약 2% 향상한 걸 알 수 있다.

2. MISSION 2-3. 수행 과정 및 결과

2-3-1. 초기 하이퍼 파라미터 세팅 1

```
## L2 Regularization도 포함
optimizer = optim.SGD(model_2.parameters(), lr=0.01, weight_decay=0.001, momentum=0.9) # 옵티마이저 정의 ->
scheduler = ReduceLROnPlateau(optimizer, mode='min', factor=0.4, patience=5, verbose=True)
scheduler_train = ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=5, verbose=True)
```

```
validation_accuracy=validate_epoch(model_2, trainloader_val, device)
print(validation_accuracy) # Validation accuracy print
```

measuring accuracy: 100%  66/66 [00:14<00:00, 4.60it/s]

64.84040069580078

Lr을 0.01, lr = 0.1로만 바꿨을 때이다. Lr을 바꾸는 것이 좀 더 dominant하다.

2. MISSION 2-3. 수행 과정 및 결과

2-3-2. 초기 하이퍼 파라미터 세팅 2

```
## L2 Regularization도 포함
optimizer = optim.SGD(model_2.parameters(), lr=0.005, weight_decay=0.001, momentum=0.9) # 옵티마이저 정의
scheduler = ReduceLROnPlateau(optimizer, mode='min', factor=0.5, patience=5, verbose=True)
scheduler_train = ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=5, verbose=True)
```

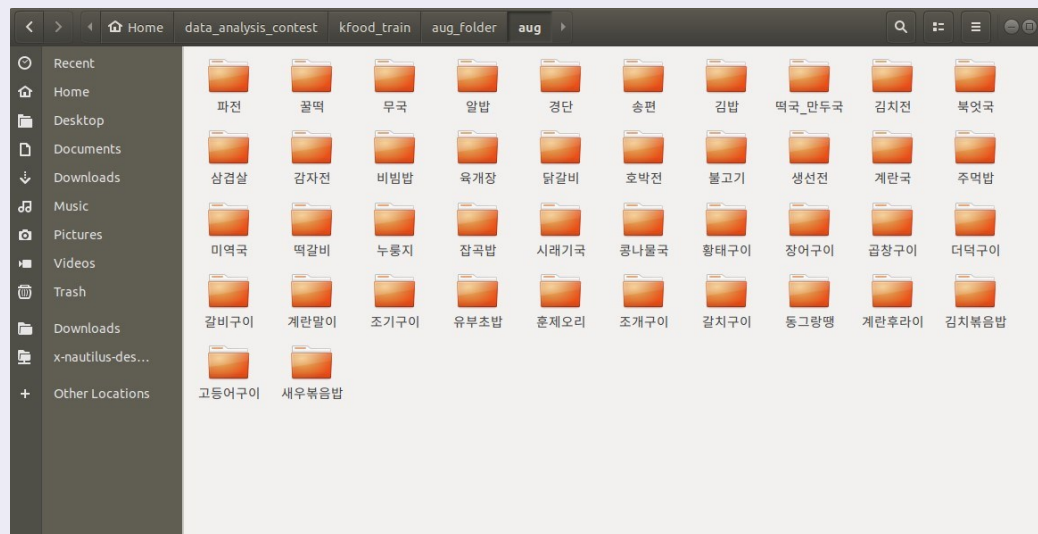
```
69.0090560913086
```

Augmentation 오류를 해결한 뒤에는 최종적으로는 lr을 0.005로 설정하여 최종적으로는 69%의 accuracy를 얻을 수 있었다.

2. MISSION 2 고찰

Augmentation

1. Augmentation 문제 : 초기에 Augmentation을 한 파일들을 한 파일에 저장했었다. 현재 imageFolder를 사용하여 data를 불러오는 방식을 사용하고 있었는데, kfood_train은 이미 폴더명이 클래스로 구분되어 있기 때문에 따로 label을 지정해주지 않아도 모델이 학습할 수 있었다. 따라서, augmentation 폴더도 각 클래스 별로 생성하여 만들어서 해결하였다.



3. MISSION 3 – 문제 및 데이터 이해

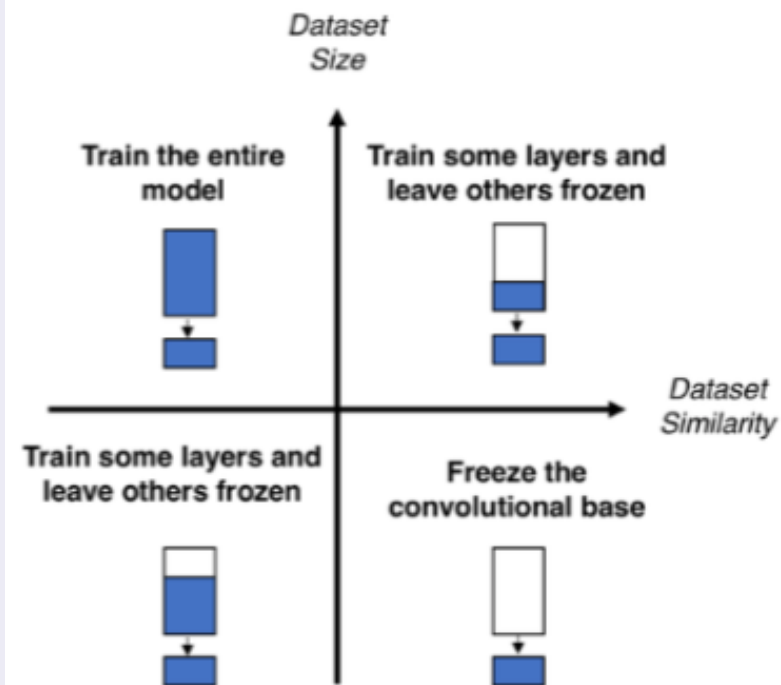
문제의 이해

- MISSION 3의 3-1은 Mission 2에서 학습시킨 모델의 가중치와 bias를 사용하여 건강관리를 위한 음식 이미지 데이터를 13개의 클래스로 분류하는 모델을 만들고 학습시킨다. 이 과정에서 다양한 방법을 시도하여 학습시키고 가장 우수한 성능을 보이는 setting을 찾는다. MISSION 1, 2처럼 새로운 RESNET18 MODEL을 선언하고 학습시키는 것이 아닌 MISSION 2 MODEL을 가져와 학습시켜야 한다. 때문에 MISSION 2 MODEL mission2.pt를 mission3.ipynb에 LOAD하는 것이 중요하다.
- MISSION 3의 3-2는 가장 높은 정확도의 모델 setting을 찾기 위하여 수행한 과정과 결과를 논리적으로 따지고 최종 결과를 도출해낸다.

3. MISSION 3 – 문제 및 데이터 이해

문제의 이해

MISSION 3에서 MISSION 2의 모델을 활용하여 학습시키기 위한 방법을 모색하였고, 건강한 음식 이미지 데이터가 일반 음식 이미지 데이터에 비해 데이터 양이 적고 비슷하다고 판단하여 finetuning 기법을 사용하기로 하였다. 이는 finetuning 기법의 전략 4 (Quadrant 4)에 해당하므로 fc layer를 제외한 나머지 층을 freeze 시키고 transfer learning으로 이전 model의 가중치와 bias를 사용하기로 하였다. 따라서 MISSION 2 모델을 사용하여 fc layer만 변경하고 학습시킨다.



3. MISSION 3-1. 수행 과정 및 결과

3-1-1. Mission 2 학습 MODEL 불러오기

```
# Local File Path(model_2의 path)
path = 'C:/Users/kfood_train/train'
label_list = os.listdir(path)
PATH_2 = 'mission2.pt'
```

```
model_3 = models.resnet18()
fc_in_features = model_3.fc.in_features
model_3.fc = nn.Linear(fc_in_features, len(label_list))
model_3 = model_3.to(device)
model_3.load_state_dict(torch.load(PATH_2))
```

MISSION 2 학습 MODEL의 파일명을 PATH_2에 저장한다. MISSION 3의 학습에 사용될 model_3에 .load_state_dict()를 통하여 MISSION 2 학습 MODEL의 가중치 및 bias를 LOAD해준다. 이때 model_3의 output_features를 MODEL2의 output_features와 일치시켜야 한다. RESNET18 구조의 새로운 학습 모델 model_3를 선언하여 위 규칙을 지켜 model_2의 bias, 가중치를 Load하였다.

3. MISSION 3-1. 수행 과정 및 결과

3-1-1. Mission 2 학습 MODEL 불러오기

```
for param in model_3.parameters():  
    param.requires_grad = False
```

```
fc_in_features = model_3.fc.in_features  
model_3.fc = nn.Linear(fc_in_features, len(label_list_3))  
model_3 = model_3.to(device)
```

MISSION 2 MODEL을 Load한 뒤, model_3의 RESNET18 fc layer를 제외한 모든 신경망을 변화가 불가능하도록 고정시키고, fc layer만을 수정할 수 있도록 설정한다. 그 뒤, fc layer의 output_features의 개수를 Mission 3의 class 개수인 13개가 되도록 설정한다.

3. MISSION 3-1. 수행 과정 및 결과

3-1-1. Mission 2 학습 MODEL 불러오기

```
<All keys matched successfully>
```

```
model_3.to(device)
```

```
(fc): Linear(in_features=512, out_features=13, bias=True)
```

MISSION2 Model의 모든 가중치 및 Bias가 model_3에 성공적으로 match되는 것을 확인하였다.

그 후 model_3의 out_features를 kfood_health의 class 개수에 일치시키도록 설정된 것을 model_3.to(device)를 수행했을 때 fc layer의 out_features가 Mission3의 클래스 개수 13과 일치함을 확인하였다.

3. MISSION 3-1. 수행 과정 및 결과

3-1-1. Mission 2 학습 MODEL 불러오기

```
<All keys matched successfully>
```

```
model_3.to(device)
```

```
(fc): Linear(in_features=512, out_features=13, bias=True)
```

MISSION2 Model의 모든 가중치 및 Bias가 model_3에 성공적으로 match되는 것을 확인하였다.

그 후 model_3의 out_features를 kfood_health의 class 개수에 일치시키도록 설정된 것을 model_3.to(device)를 수행했을 때 fc layer의 out_features가 Mission3의 클래스 개수 13과 일치함을 확인하였다.

3. MISSION 3-1. 수행 과정 및 결과

3-1-2. Training 수행

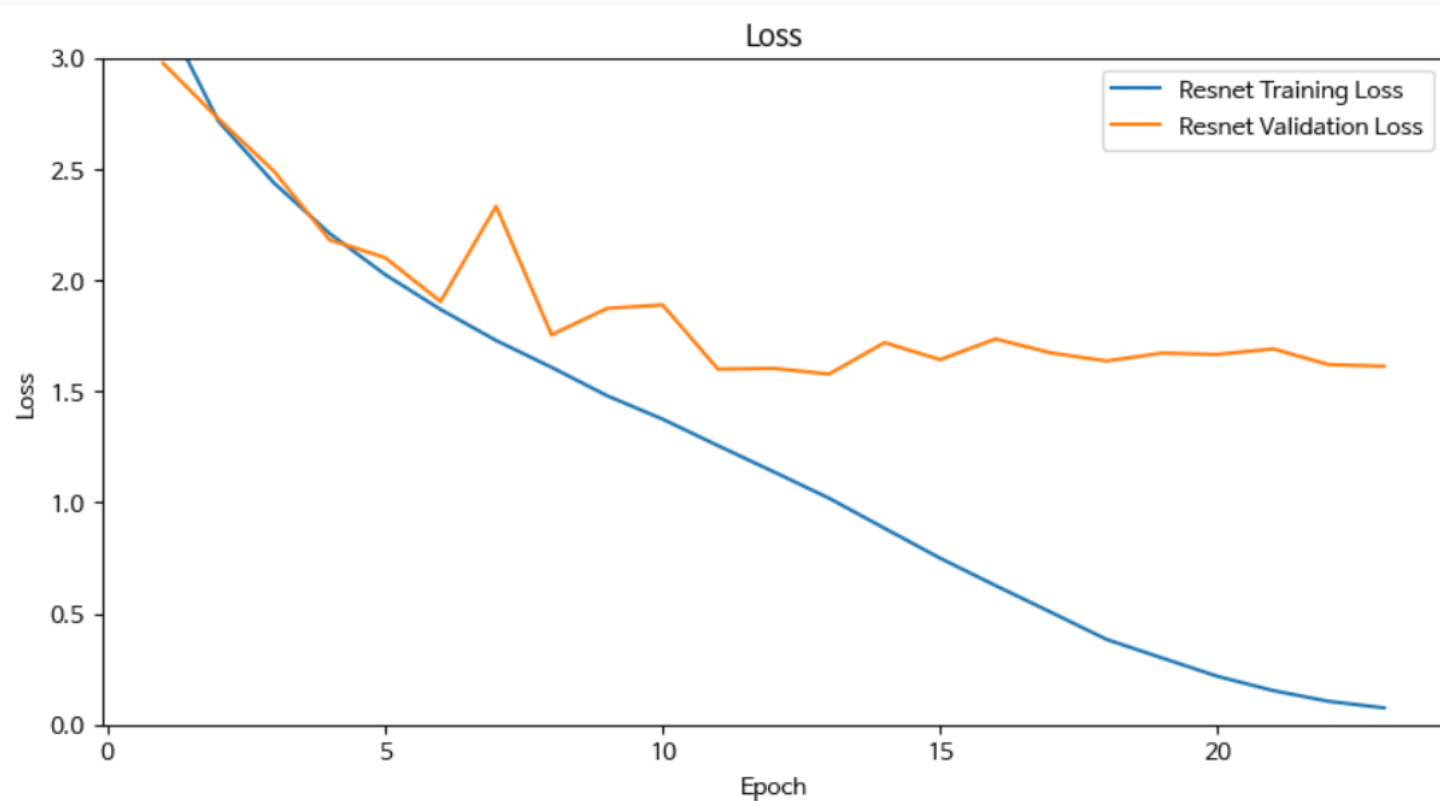
MISSION 2에서 불러온 모델을 활용하여 건강한 한국 음식 데이터 셋을 epoch 50으로 학습시킨 모습이다.

```
Epoch [1/50], Training Loss: 1.6638
Epoch [1/50], Validation Loss: 1.3737
Best parameter save model update
Epoch [2/50], Training Loss: 1.3082
Epoch [2/50], Validation Loss: 1.2939
Best parameter save model update
Epoch [3/50], Training Loss: 1.2508
Epoch [3/50], Validation Loss: 1.2568
Best parameter save model update
Epoch [4/50], Training Loss: 1.2263
Epoch [4/50], Validation Loss: 1.2282
Best parameter save model update
Epoch [5/50], Training Loss: 1.2136
Epoch [5/50], Validation Loss: 1.2198
Best parameter save model update
Epoch [6/50], Training Loss: 1.2088
Epoch [6/50], Validation Loss: 1.2141
Best parameter save model update
Epoch [7/50], Training Loss: 1.1932
Epoch [7/50], Validation Loss: 1.2028
Best parameter save model update
Epoch [8/50], Training Loss: 1.1881
Epoch [8/50], Validation Loss: 1.2049
Current Counter: 1
Epoch [9/50], Training Loss: 1.1843
...
Best parameter save model update
Epoch [50/50], Training Loss: 1.0846
Epoch [50/50], Validation Loss: 1.1062
Best parameter save model update
```

3. MISSION 3-1. 수행 과정 및 결과

3-1-2. Training 수행

학습을 통해서 출력된 loss를 plot 하여 경향성을 쉽게 파악하도록 하였고 그 결과, training에 대한 loss는 잘 나왔으나 validation에 대한 loss는 5 epoch부터 oscillation 하는 모습을 볼 수 있고 최종 loss 결과가 다소 높은 것을 확인할 수 있다.



3. MISSION 3-2. 수행 과정 및 결과

성능 분석

```
validation_accuracy=validate_epoch(model_3, trainloader_val_3, device)
print(validation_accuracy) # Validation accuracy print
```

```
61.28118133544922
```

최종적으로 MISSION 2에서 가져온 model에 transfer learnin을 적용하고 50 epoch 만큼 학습시켜 validation 데이터 셋으로 정확도를 측정한 결과 약 61.28% 측정되었다. MISSION 2의 정확도에 비해 낮은 것을 확인할 수 있었다. MISSION 2에서 업데이트 한 가중치와 bias 값이 MISSION 3 데이터 셋의 특징 추출을 하는 데에 매우 부합하지는 않았음을 알 수 있고 이에 따라 MISSION 2에서 하이퍼 파라미터를 더 잘 조절하여 업데이트 된 가중치를 사용 했더라면 더 높은 정확도를 얻을 수 있었을 것이라고 생각한다.

3. MISSION 3 고찰

linear probing, few shot learning

1. MISSION 3에서 학습 방법을 선정과정에서 Linear probing과 few shot learning을 적용하지 않는 것이 낫다고 판단하였다. Linear probing의 경우 딥러닝에서 transfer learning 하는 방법 중 일부로 이미 transfer learning을 적용시켰으므로 제외하였고 few shot learning의 경우 매우 제한된 수의 학습 샘플로 클래스를 학습하는 방법인데 가지고 있는 건강 음식 이미지 데이터 셋은 일반 음식 이미지 데이터 셋에 비해 적긴 하지만 매우 제한적인 수가 아니기 때문에 MISSION 3 모델 학습 방법에서 제외하였다. MISSION 3의 최종 정확도가 생각보다 낮게 나왔는데 이 때 처음에 데이터셋이 작고 유사하다고 판단하여 전략 4번을 사용하였다. 만약 finetuning 전략에서 2번(Quadrant 2)나 3번(Quadrant 3)을 시도하여 비교하였더라면 더 나은 성능을 기대할 수 있었을 것이다.

감사합니다.

대학부

네트로닉스

팀장 이민성

박민선

석대근

김완수

2023 데이터 크리에이터 캠프

DATA CREATOR CAMP